Solutions to Self Study 2 Exercises

CLRS4 26-1 (CLRS3 27-1)

a. Here is the algorithm expressed using nested parallelism (initial call with $l = 1$ and $r = n$):

```
SUM-ARRAYS-MT(A, B, C, l, r)
1  if l == r
2      C[l] = A[l] + B[l]
3  else
4      q = ⌊(l + r)/2⌋
5      spawn SUM-ARRAYS-MT(A, B, C, l, q)
6      SUM-ARRAYS-MT(A, B, C, q + 1, r)
7      sync
```

The work of the algorithm is $\Theta(n)$, the span is $\Theta(\lg n)$, so the parallelism is $\Theta(n/\lg n)$.

b. When *grain-size* $= 1$, the parallelism is just 1, because the work and the span are both $\Theta(n)$.

c. Here is the analysis. The work is $\Theta(n)$, independent of the *grain-size*. The span, as a function of *grain-size*, is $S(\textit{grain-size}) = n/\textit{grain-size} + \textit{grain-size}$. To minimize the span, we differentiate it and solve the equation $S'(\textit{grain-size}) = 0$. This gives that *grain-size* $= \sqrt{n}$ minimizes the span. The span is then $n/\sqrt{n} + \sqrt{n} = \sqrt{n} + \sqrt{n} = \Theta(\sqrt{n})$. The parallelism is work divided by span. Thus, when *grain-size* $= \sqrt{n}$, the parallelism is:

$$P = \frac{n}{\sqrt{n}} = \Theta(\sqrt{n}).$$

Comparing this with the parallelism in part *a*, we see that $n/\lg n = \omega(\sqrt{n})$ (To see it, observe that $\sqrt{n} = n/\sqrt{n}$ and remember that $\sqrt{n}$, as a polynomial function, dominates a logarithm). So the moral is that the standard "implementation" of parallel loops (as in part *a*) is the best way to parallelize loops using nested parallelism.

CLRS4 16-2 (CLRS3 17-2)

In the following $k = \lceil \lg(n+1) \rceil$ (as defined in the exercise).

a. As each of the arrays $A_0, A_1, \ldots, A_{k-1}$ is sorted, we can perform a binary search in each of them to implement the SEARCH operation. In the worst case, all of the arrays are full and the search operation is unsuccessful, which means that the search in the array A_i costs $\Theta(\lg 2^i) = \Theta(i)$. Summing up, the total running time is

$$\Theta\left(\sum_{i=0}^{k-1} i\right) = \Theta(k(k-1)/2) = \Theta(\lceil \lg(n+1) \rceil (\lceil \lg(n+1) \rceil - 1)/2) = \Theta(\lg^2 n).$$

b. Here is the algorithm for insertion:

INSERT(x)

1 Let B be an array containing just x
2 $i = 0$
3 **while** A_i is not empty
4 Merge B and A_i and store the result in B
5 $i = i + 1$
6 Replace A_i with B

Merge in line 4 is the merge from the merge sort. It merges two sorted arrays into one. Note that the two merged arrays, B and A_i, have exactly the same number of elements—the size of B doubles in each iteration. The running time of the merge is linear, more precisely $2m$, where m is the size of B.

In the worst case, all arrays $A_0, A_1, \ldots, A_{k-2}$ are full and they are merged into A_{k-1} in a series of $k-1$ merges. The total running time of INSERT is

$$2\sum_{i=0}^{k-2} 2^i \;=\; 2(2^{k-1}-1) \;=\; 2^k - 2 \;=\; \Theta(2^{\lceil \lg(n+1)\rceil}) \;=\; \Theta(n).$$

To get the amortized cost, here is an aggregate analysis. Consider an initially empty structure and a sequence of n insertions. A_0 is merged every second insertion ($n/2$ times), with 2 being the cost of merging. A_1 is merged every fourth insertion ($n/4$ times), with 4 being the cost of merging. A_i is merged $n/2^{i+1}$ times, with 2^{i+1} being the cost of merging. After n insertions, all arrays except the last one (A_{k-1}) have been merged at least once and the total cost of all merges is

$$\sum_{i=0}^{k-2} \frac{n}{2^{i+1}} 2^{i+1} \;=\; (k-1)n \;=\; (\lceil \lg(n+1)\rceil - 1)n \;=\; \Theta(n \lg n).$$

Thus, the amortized cost of one insertion is $\Theta(\lg n)$.

This can also be seen using the accounting method. Consider how a specific element moves in the structure. An element always moves from a smaller array to a larger array, and this happens only when the array that contains the element is merged. Thus, an element can participate in at most $k-1$ merges after n insertions. The cost of merging *per element* is $O(1)$. Thus, paying k dollars for insertion covers all the costs. One dollar is used for line 1 of INSERT and $k-1$ dollars are put into the account of the element to pay for all the future merges this element will be involved in. The amortized cost of an insertion in a sequence of n insertions is therefore $\Theta(k) = \Theta(\lg n)$.

c. Here is a sketch of the DELETE(x) (x is a pointer to an element in the structure):

1. Let A_j be the smallest full array in the structure and let y be the last element of A_j.
2. Let A_i be the array that contains x.
3. Remove x from A_i. If x was not y, remove y from A_j, insert y into A_i, and move it to the correct place in A_i.
4. A_j has now $2^j - 1$ elements. "Unmerge" its elements into arrays $A_0, A_1, \ldots, A_{j-1}$: the first element is put into A_0, the next two elements into A_1, the next four elements into A_2, and so on. A_j is left empty. Note, that all the arrays $A_0, A_1, \ldots A_{j-1}$ end up already sorted, because A_j was sorted and we did not shuffle its elements.

The running time of DELETE is dominated by steps 3 and 4. Each of them takes $\Theta(n)$ time in the worst case, which happens when $i = k - 1$ and $j = k - 2$. Step 3 takes $\Theta(2^i) = \Theta(n)$ time to move y into its correct place in A_i and step 4 takes $\Theta(2^j) = \Theta(n)$ time to "unmerge" the array A_j.

The amortized cost of an operation in a sequence of n insertions and deletions is also $\Theta(n)$. This can be shown by doing the aggregate analysis on the following worst-case sequence of operations. Let $n/2$ be a power of 2. Do $n/2$ insertions followed by $n/4$ pairs of DELETE and INSERT. The initial $n/2$ insertions take $\Theta(n \lg n)$ time and create a single full array (because $n/2$ is a power of 2). For each of the following DELETE-INSERT pairs, DELETE takes $\Theta(n)$ time to "unmerge" the full array and INSERT takes $\Theta(n)$ time to merge all the arrays back into one. The total running time of all operations in this sequence is $\Theta(n \lg n) + 2 \cdot \Theta(n) \cdot n/4 = \Theta(n^2)$. Thus, the amortized cost of a single operation is $\Theta(n)$, not better than the worst-case cost.